

유향했던 클로드코드 도구들

크롬 개발자도구 MCP · LSP 플러그인 · Hook

발표자: 이주엽

목차

1. 크롬 개발자도구 MCP — 브라우저를 Claude의 눈으로
2. LSP 플러그인 — 실시간 코드 진단
3. Hook — Claude의 행동을 내 입맛대로

1. 크롬 개발자도구 MCP

크롬 개발자도구 MCP란?

MCP 연결 후 Claude Code가 크롬 브라우저를 직접 열고 제어할 수 있다

> 크롬개발자도구 MCP로 `http://localhost:5174/`에 접속해봐

- Calling `plugin:chrome-devtools-mcp:chrome-devtools 2 times...` (ctrl+o to expand)

"크롬개발자도구 MCP로 `http://localhost:5174/`에 접속해봐" → Claude가 직접 브라우저를 열고 페이지를 확인

활용 1 — 디자인 수정 지시

```
> "링크 복사" 버튼이 너무 배경이랑 구분이 안돼. 눈에 잘 띄게 흰색, 누르면 검은색으로 바뀌게 해줘

Read 1 file (ctrl+o to expand)

• Update(frontend\src\pages\EventDetail.jsx)
  | Added 1 line, removed 1 line
  | 387     <div className="flex gap-2">
  | 388       <button
  | 389         onClick={copyLink}
  | 390         className="flex items-center gap-1.5 px-4 py-2 rounded-xl border border-gray-200 text-sm font-medium text-gray-600 hover:bg-gray-50 transition-colors"
  | 391         className={flex items-center gap-1.5 px-4 py-2 rounded-xl border text-sm font-medium transition-colors ${copied ? 'bg-gray-900 border-gray-800 text-white' : 'bg-white border-gray-300 text-gray-700 hover:bg-gray-50'}}
  | 392       >
  | 393       <svg className="w-3.5 h-3.5" fill="none" viewBox="0 0 24 24" stroke="currentColor">
  | 394         <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M8 16H6a2 2 0 0 2-2V6a2 2 0 0 2 2V8z" />
  |
  | • 버튼이 보이지 않아 위로 스크롤하겠습니다.
  |
  | • "링크 복사" 버튼이 흰 배경으로 라벤더 배경과 분리되어 잘 보입니다. 클릭하면 검은 배경으로 바뀝니다. 더 바꿀 부분 있으면 말씀해주세요.
```

- "링크 복사" 버튼이 너무 배경이랑 구분이 안돼. 눈에 잘 띄게 흰색, 누르면 검은색으로 바뀌게 해줘"
- Claude가 직접 페이지를 보고 → 코드를 바로 수정
- 사진을 줄 필요 없이, 말로만 해도 정확하게 전달

활용 2 — 개발자도구 디버깅

기존 방식 (F12)

1. 개발자도구 직접 열기
2. Console / Network 탭 확인
3. 오류 발견
4. Claude에게 복붙해서 설명
5. 원인 파악 요청...

→ 뭐가 문제인지 내가 먼저 찾아야 함

MCP 방식

1. "이 페이지 열어서 Console이랑 Network 봐줘"
2. Claude가 직접 개발자도구 접근
3. 원인 분석 → 해결방안 제시 자동화

→ 지시만 하면 문제 진단부터 수정까지 빠르게

"뭐가 문제인지 설명할 필요가 없어졌다."

— 그냥 보라고 하면 되니까

2. LSP 플러그인

LSP가 뭔지는 이미 알고 있다

코딩할 때 보이는 빨간 밑줄 — 그게 LSP다

- **Language Server Protocol** — VS Code, JetBrains 등 IDE에 내장된 코드 진단 엔진
- 타입 오류, 참조 오류, 린트 경고를 **실시간으로** 잡아준다
- IDE가 이걸 화면에 그려주는 것뿐, 진단 자체는 LSP가 한다

Claude Code는 코드를 어떻게 볼까?

Claude의 기본 방식

- 코드를 텍스트로 읽는다
- 오류를 찾으려면 관련 파일을 전부 읽어야 함
- 프로젝트가 커질수록 → 토큰 낭비 심해짐
- 파일을 다 읽어도 → 진단이 부정확할 수 있음

LSP 플러그인 사용 시

- IDE가 이미 분석한 **진단 결과만** 받아온다
- 파일을 직접 다 읽을 필요 없음 → **토큰 절약**
- LSP가 정확히 짚어준 위치·원인 → **더 정확한 진단**

실제 동작 — 더블체크

코드를 수정한 뒤, Claude가 **알아서 LSP로 검증한다**

- LSP `documentSymbol` 로 파일 구조 파악
- 282개 심볼 분석 → 진단 이슈 2건 발견
- 파일을 전부 읽지 않고도 **정확하게 문제 위치** 특정
- 분석 완료 후 취약점을 심각도 순으로 정리

```
• 주요 백엔드 파일들을 LSP로 분석하겠습니다.

• LSP(operation: "documentSymbol", file:
  "backend\src\routes\auth.js")
  └ Found 282 symbols (ctrl+o to expand)

• LSP(operation: "documentSymbol", file:
  "backend\src\middleware\auth.js")
  └ Error performing documentSymbol: Failed to
    sync file open C:\Users\yeop7382\yeop_progra
    m\festicket\backend\src\middleware\auth.js:
    Cannot send notification to LSP server
    'plugin:vtls:typescript': server is
    starting

Read 2 files (ctrl+o to expand)
└ Found 2 new diagnostic issues in 1 file (ctrl+o
  to expand)

• 비밀번호 재설정과 결제 라우트도 확인합니다.

Read 2 files (ctrl+o to expand)

• 분석 완료했습니다. 발견된 취약점을 심각도 순으로
  정리합니다.
```

토큰은 아끼고, 정확 도는 높이고

LSP는 Claude가 코드를 이해하는 방식 자체를 바꿔준다

3. Hook

Hook이란?

Claude Code가 도구를 실행하는 순간에 내 쉘 명령을 끼워넣는 기능

이벤트

시점

PreToolUse

도구 실행 직전

PostToolUse

도구 실행 직후

Notification

Claude가 알림을 보낼 때

Stop

Claude가 응답을 멈출 때

```
// .claude/settings.json
{
  "hooks": {
    "PostToolUse": [{ "matcher": "Write|Edit", "hooks": [{ "type": "command", "command": "prettier --write $FILE" }] }]
  }
}
```

개인적으로 느낀 한계 / 불편함 / 개선점

- 설치 귀찮음 — MCP 서버, LSP 플러그인, Hook 설정 등 초기 세팅이 번거롭다
- 관리 복잡 — 프로젝트마다 설정이 달라질 수 있고, 업데이트 시 깨지는 경우가 있다

감사합니다

질문 있으신가요?